DSA MASTERY LESSON 1

# The Array

## The simplest data structure, and the foundation of almost every other one

Real-world scenario, memory diagrams, five core operations and working code in Python, Java, Go, Rust, JavaScript and TypeScript

**O(1)**
Access by index, instant at any size

**O(n)**
Search, insert, delete in the middle

**6**
Languages, every example tested


Lockers 2 | Memory 3 | Operations 3
Cheat sheet 4 | Code 5 | Practice 9


LEVEL: BEGINNER PREREQUISITES: NONE

Start here: next come the Two-Pointer technique and Linked Lists. Complexity details live in the separate Complexity Handbook.

WHAT YOU WILL LEARN

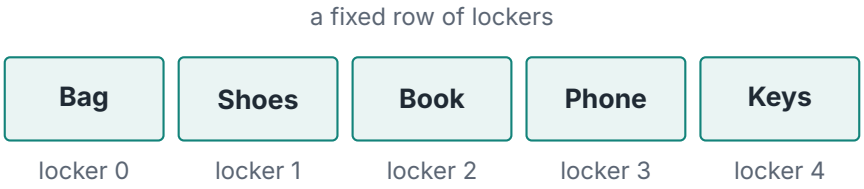
- What an array is, using a real-world analogy.
- How an array is laid out in computer memory, and why indexing is instant.
- The five core operations and their time complexity.
- Working code for every operation in six languages.
- Practice problems to lock in the idea.

## Contents


1 Real-World Scenario: A Row of Lockers . . . 2
2 How an Array Sits in Memory . . . 3
3 The Five Core Operations . . . 3
4 Complexity Cheat Sheet . . . 4
5 The Code, in Six Languages . . . 5
6 Practice Problems . . . 9
7 What Comes Next . . . 9


## 1 Real-World Scenario: A Row of Lockers

A gym has **5 lockers in a row**, side by side, numbered from 0.




Three properties make this an array:

1. **Fixed size.** The row has exactly 5 lockers.
2. **Side by side.** The lockers are *contiguous*, with no gaps.
3. **Numbered.** To reach locker 3 you walk straight to it. You do not open lockers 0, 1 and 2 first.

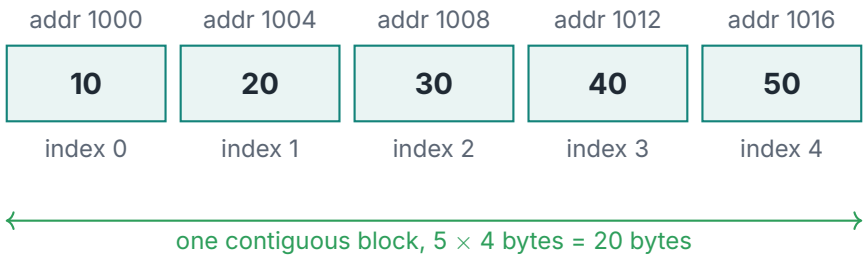
DEFINITION

An **array** is a collection of items stored in **contiguous memory**, where each item is accessed by its **index**.

**More real-world arrays:** a music playlist (song #3 is `playlist[2]`), the days of the week, the pixels of an image (a 2D array), and 24 hourly temperature readings (`temps[14]` is the 2 PM reading).

## 2 How an Array Sits in Memory

RAM is a giant row of numbered boxes called *addresses*. An array of 5 integers, each taking 4 bytes, occupies one unbroken block:



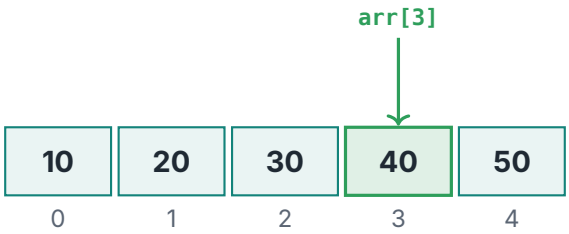

The computer remembers only the **start address** (1000). To find any element it uses one formula:

> **THE ADDRESS FORMULA**
>
> ```
> address of arr[i] = start_address + (i × size_of_each_element)
> ```
>
> Example: `arr[3]` → 1000 + (3 × 4) = **1012**

This is a single calculation, with no searching. Reading by index is therefore **O(1)**: the cost does not grow with the size of the array. It takes the same time for 5 elements or 5 million.

## 3 The Five Core Operations

### Access by index — O(1)




We jump straight to the cell, with no scanning.

### Search for a value — O(n)

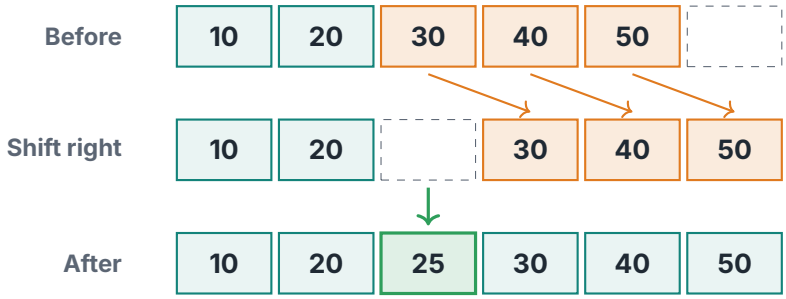
If you do not know where `40` is, you must check each cell in turn.




In the *worst case* (the target is last, or missing) we inspect all $n$ items, so the cost is **O(n)**.

### Insert in the middle — O(n)

Insert `25` at index 2. Everything from index 2 onward must shift one place right to make room.




Think of lockers bolted together: you cannot squeeze one in, so you have to move the rest over. Inserting at the *front* is the worst case, because all $n$ items shift.

### Delete from the middle — O(n)

Deleting leaves a hole, so the elements after it shift left to close the gap.




### Append at the end — O(1)*

If there is free space at the end, nothing needs to move.




* **Amortized.** A *dynamic array* (Python `list`, Java `ArrayList`, Go slice, Rust `Vec`, JavaScript array) allocates a bigger block when it fills up and copies everything across. That resize costs **O(n)**, but it happens rarely. Averaged over many appends, each one is **O(1)**.

## 4 Complexity Cheat Sheet

| Operation | Time | Why |
|---|---|---|
| Access `arr[i]` | O(1) | Address formula, one calculation |
| Search (unsorted) | O(n) | Check one by one |
| Insert at the end | O(1)* | No shifting (amortized) |
| Insert at start or middle | O(n) | Shift elements right |
| Delete at start or middle | O(n) | Shift elements left |

**Strengths**

- Very fast reads by index
- Simple to understand and use
- Cache-friendly (contiguous data)

**Weaknesses**

- Slow insert/delete in the middle
- Fixed size (plain arrays)
- Search is **O(n)** unless sorted

# 5 The Code, in Six Languages

Every program below does the same five things on the array `[10, 20, 30, 40, 50]`: access index 3, linearly search for 40, insert 25 at index 2, delete index 2, and append 60. All six programs were compiled and run, and each printed the same results:

```
40                      access
3                       search result
[10, 20, 25, 30, 40, 50]  after insert
[10, 20, 30, 40, 50]      after delete
[10, 20, 30, 40, 50, 60]  after append
```

(Go prints lists without commas and Rust prints the search result as `Some(3)`. The values are the same.)

## Java

**Java: `int[]` and `ArrayList`**

```java
import java.util.ArrayList;
import java.util.Arrays;

public class ArraysDemo {
    static int linearSearch(int[] a, int target) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] == target) return i;
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};

        // 1. Access by index: O(1)
        System.out.println(arr[3]);                // 40

        // 2. Linear search: O(n)
        System.out.println(linearSearch(arr, 40));  // 3

        // Fixed-size int[] cannot grow, so use ArrayList for the rest
        ArrayList<Integer> list = new ArrayList<>(Arrays.asList(10, 20, 30, 40, 50));

        // 3. Insert 25 at index 2: O(n)
        list.add(2, 25);
        System.out.println(list);   // [10, 20, 25, 30, 40, 50]

        // 4. Delete the element at index 2: O(n)
        list.remove(2);
        System.out.println(list);   // [10, 20, 30, 40, 50]

        // 5. Append at the end: O(1) amortized
        list.add(60);
        System.out.println(list);   // [10, 20, 30, 40, 50, 60]
    }
}
```

## Python

**Python 3:** `list`

```python
arr = [10, 20, 30, 40, 50]

# 1. Access by index: O(1)
print(arr[3])                # 40

# 2. Linear search: O(n)
def linear_search(a, target):
    for i in range(len(a)):
        if a[i] == target:
            return i
    return -1

print(linear_search(arr, 40)) # 3

# 3. Insert 25 at index 2: O(n)
arr.insert(2, 25)
print(arr)                   # [10, 20, 25, 30, 40, 50]

# 4. Delete the element at index 2: O(n)
arr.pop(2)
print(arr)                   # [10, 20, 30, 40, 50]

# 5. Append at the end: O(1) amortized
arr.append(60)
print(arr)                   # [10, 20, 30, 40, 50, 60]
```

## JavaScript

**JavaScript (Node.js)**

```javascript
const arr = [10, 20, 30, 40, 50];

// 1. Access by index: O(1)
console.log(arr[3]);                // 40

// 2. Linear search: O(n)
function linearSearch(a, target) {
  for (let i = 0; i < a.length; i++) {
    if (a[i] === target) return i;
  }
  return -1;
}
console.log(linearSearch(arr, 40));  // 3

// 3. Insert 25 at index 2: O(n)
arr.splice(2, 0, 25);
console.log(arr);                   // [10, 20, 25, 30, 40, 50]

// 4. Delete the element at index 2: O(n)
arr.splice(2, 1);
console.log(arr);                   // [10, 20, 30, 40, 50]

// 5. Append at the end: O(1) amortized
arr.push(60);
console.log(arr);                   // [10, 20, 30, 40, 50, 60]
```

## Go

**Go: slice**

```go
package main

import "fmt"

func linearSearch(a []int, target int) int {
    for i := range a {
        if a[i] == target {
            return i
        }
    }
    return -1
}

func main() {
    arr := []int{10, 20, 30, 40, 50}

    // 1. Access by index: O(1)
    fmt.Println(arr[3]) // 40

    // 2. Linear search: O(n)
    fmt.Println(linearSearch(arr, 40)) // 3

    // 3. Insert 25 at index 2: O(n)
    arr = append(arr[:2], append([]int{25}, arr[2:]...)...)
    fmt.Println(arr) // [10 20 25 30 40 50]

    // 4. Delete the element at index 2: O(n)
    arr = append(arr[:2], arr[3:]...)
    fmt.Println(arr) // [10 20 30 40 50]

    // 5. Append at the end: O(1) amortized
    arr = append(arr, 60)
    fmt.Println(arr) // [10 20 30 40 50 60]
}
```

## TypeScript

**TypeScript**

```typescript
const arr: number[] = [10, 20, 30, 40, 50];

// 1. Access by index: O(1)
console.log(arr[3]);                 // 40

// 2. Linear search: O(n)
function linearSearch(a: number[], target: number): number {
  for (let i = 0; i < a.length; i++) {
    if (a[i] === target) return i;
  }
  return -1;
}
console.log(linearSearch(arr, 40));  // 3

// 3. Insert 25 at index 2: O(n)
arr.splice(2, 0, 25);
console.log(arr);                    // [10, 20, 25, 30, 40, 50]

// 4. Delete the element at index 2: O(n)
arr.splice(2, 1);
console.log(arr);                    // [10, 20, 30, 40, 50]

// 5. Append at the end: O(1) amortized
arr.push(60);
console.log(arr);                    // [10, 20, 30, 40, 50, 60]
```

## Rust

**Rust:** `Vec<i32>`

```rust
fn linear_search(a: &[i32], target: i32) -> Option<usize> {
    for (i, &x) in a.iter().enumerate() {
        if x == target {
            return Some(i);
        }
    }
    None
}

fn main() {
    let mut arr = vec![10, 20, 30, 40, 50];

    // 1. Access by index: O(1)
    println!("{}", arr[3]); // 40

    // 2. Linear search: O(n)
    println!("{:?}", linear_search(&arr, 40)); // Some(3)

    // 3. Insert 25 at index 2: O(n)
    arr.insert(2, 25);
    println!("{:?}", arr); // [10, 20, 25, 30, 40, 50]

    // 4. Delete the element at index 2: O(n)
    arr.remove(2);
    println!("{:?}", arr); // [10, 20, 30, 40, 50]

    // 5. Append at the end: O(1) amortized
    arr.push(60);
    println!("{:?}", arr); // [10, 20, 30, 40, 50, 60]
}
```

### LANGUAGE NOTES

- **Java:** a plain `int[]` has a fixed length and cannot grow, so insert, delete and append use `ArrayList`, Java's dynamic array.
- **Go:** there is no built-in insert or delete. You build them with `append` and slicing, which makes the **O(n)** shifting visible.
- **Rust:** `linear_search` returns an `Option<usize>`: `Some(index)` if found, `None` if not, so the compiler forces you to handle "not found" instead of returning a magic $-1$.
- **TypeScript:** identical to JavaScript except for the type annotations (`number[]`, `: number`), which are checked at compile time.

## 6 Practice Problems

Try these before the next lesson, in any of the six languages.

1. **Easy.** Given `[4, 9, 1, 7]`, find the largest number without using a built-in `max`.
2. **Easy.** Reverse an array in place: `[1,2,3,4,5]` → `[5,4,3,2,1]`.
   Hint: put one pointer at each end and swap as they move toward the middle.
3. **Medium.** Given `[3, 8, 2, 5]` and a target `10`, find two numbers that add up to the target. Describe your approach and its time complexity.

CHECK YOUR UNDERSTANDING

1. Why is `arr[i]` **O(1)**, but searching for a value **O(n)**?
2. Why is inserting at the *front* of an array expensive?
3. Why can appending to a dynamic array be called **O(1)** even though a resize costs **O(n)**?

## 7 What Comes Next

**Lesson 2: the Two-Pointer technique.** It solves problems like practice #2 and #3 efficiently. After that, **Linked Lists**, a structure built to fix the array's slow-insertion weakness.